UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

AED - Proyecto 2



Algoritmos y Estructuras de Datos (CS2023)

Teoría 2

Ciclo euleriano: el algoritmo de Hierholzer, animado

Nombre y Apellido	Código de Estudiante	Contribución
Benites Camacho, Alonso Aarón	202410142	Implementación en C++ del algoritmo y generación de la traza.
Ucañani Tintaya, Miguel Fernando	202310383	Explicación conceptual e informe.
Ortiz Palomino, David Alonso	201910408	Guión, escena de Manim y diseño de la animación (video).

Profesor:
Romero Calla, Luciano Arnaldo

2026 - II

Repositorio: https://github.com/BecamC/Proyecto-2-AED.git

Video: [enlace a YouTube no listado / Drive]

1. Descripción del algoritmo y su ADT

Tema asignado: ciclo euleriano. Un ciclo euleriano es un ciclo que recorre cada arista de un grafo exactamente una vez y regresa al vértice de partida. El problema nació con los puentes de Königsberg (Euler, 1736). Un grafo no dirigido admite un ciclo euleriano si y solo si (i) todos sus vértices tienen grado par y (ii) todos los vértices con aristas pertenecen a una misma componente conexa. Los vértices aislados no afectan la existencia del ciclo.

Aplicaciones: rutas que deben cubrir todas las calles o tramos exactamente una vez (recolección de basura, limpieza de calles, reparto de correo: problema del cartero chino), trazado de plotters y máquinas CNC sin levantar la herramienta, y ensamblaje de ADN mediante grafos de De Bruijn.

Algoritmo de Hierholzer (1873). Permite hallar el ciclo en tiempo lineal. Se parte de un vértice con aristas y se usa una *pila* de vértices. En cada paso, sea *u* el tope: si *u* tiene una arista libre (no usada) *(u,v)*, se marca como usada y se apila *v*; si no tiene, se desapila *u* y se agrega a una salida provisional. Al vaciar la pila, la salida contiene el ciclo en orden inverso, por lo que se invierte. Intuitivamente, el algoritmo sigue una ruta hasta quedar atrapado (siempre en el vértice de partida, por la paridad de los grados) y, al retroceder, "cose" los sub-ciclos que quedaron pendientes en vértices intermedios.

ADT involucrados. El grafo no dirigido se representa con listas de adyacencia, donde cada entrada guarda (vecino, id de arista), de modo que ambas copias de una arista comparten identificador y pueden marcarse a la vez. La pila (ADT *Stack*, LIFO) controla el recorrido; un puntero por vértice (`ptr[u]`) evita reexaminar aristas ya descartadas.

```
Hierholzer(G, s):                        // G conexo y con grados pares
  pila ← [s];  salida ← []
  mientras pila no vacía:
     u ← tope(pila)
     avanzar ptr[u] mientras la arista ptr[u] esté usada
     si ptr[u] = fin:  desapilar u;  salida.agregar(u)
     sino:  (v,id) ← adj[u][ptr[u]];  usada[id] ← verdadero;  ptr[u]++;  apilar v
  retornar invertir(salida)
```

2. Implementación y animación (animación real, no simulada)

El proyecto se divide en dos capas desacopladas, de modo que la animación depende de la lógica real del algoritmo:

- **euler.cpp (C++17).** Lee el grafo por entrada estándar (`n m` y una arista por línea), verifica las dos condiciones de existencia, ejecuta Hierholzer implementado por el grupo (la pila y los punteros se programaron directamente) y escribe un JSON con el grafo, los grados, si existe ciclo y por qué, el ciclo, el orden de aristas y la lista de **eventos**.
- **animacion.py (Manim Community).** Lee ese JSON y reproduce los eventos uno a uno: dibuja el grafo, muestra los grados, mueve un marcador por cada arista consumida, y actualiza la pila y la salida provisional. No calcula recorridos propios ni inventa pasos; incluso verifica con `assert` que la salida invertida coincida con el ciclo calculado en C++.

Evento	Significado en el algoritmo	Efecto en la animación
`start`	Se apila el vértice inicial	Aparece el marcador amarillo en el vértice
`traverse`	Se consume una arista y se apila su destino	La arista se pinta de verde y el marcador avanza
`finish`	Vértice sin aristas libres: se desapila y pasa a la salida	El marcador retorna al nuevo tope (sin recorrer arista)
`reverse`	Se invierte la salida provisional	La etiqueta cambia a "Ciclo final" (verde)

Casos de prueba incluidos. (a) `grafo_ejemplo.txt`: "corbata" de 5 vértices y 6 aristas (dos triángulos que comparten el vértice 0); resultado `0→1→2→0→3→4→0`. (b) `grafo_retroceso.txt`: otra corbata centrada en el vértice 1, donde el algoritmo debe retroceder a un vértice intermedio para incorporar un sub-ciclo; resultado `0→1→3→4→1→2→0`. (c) `grafo_borde.txt`: caso de borde, un camino 0–1–2 con grados (1,2,1); el programa informa que no existe ciclo por tener vértices de grado impar y la animación se detiene mostrando el motivo en rojo. El programa también cubre grafo sin aristas (se considera sin ciclo), grafo no conexo, aristas paralelas y lazos.

3. Herramientas y pasos para reproducir el video

Herramienta	Uso
C++17 (g++)	Implementación de Hierholzer y generación de la traza JSON
Python 3 + Manim Community	Animación de la traza (escena `GrafoEuleriano`)
Git / GitHub	Control de versiones y alojamiento del código

```
1) g++ -std=c++17 -O2 -Wall -Wextra euler.cpp -o euler.exe
2) cmd /c "euler.exe < grafo_ejemplo.txt > trace.json"          (idem para los otros grafos)
3) python -m manim -pqh animacion.py GrafoEuleriano              (-pql: baja resolución)
   Otro caso (PowerShell):  $env:EULER_TRACE="trace_retroceso.json"  y repetir el paso 3
```

El video de avance se renderizó en 480p15 (modo rápido `-pql`) y dura 42 s; la versión final se renderizará en alta resolución con `-pqh`.

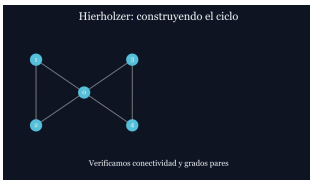
4. Análisis de complejidad

Sean *V* el número de vértices y *E* el de aristas.

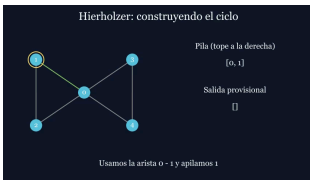
- **Verificación de existencia:** calcular grados es O(V+E) y la comprobación de conectividad (recorrido desde un vértice con aristas) es O(V+E).
- **Hierholzer:** cada arista se marca como usada una sola vez, y como `ptr[u]` solo avanza, cada entrada de las listas de adyacencia (2E en total) se examina a lo sumo una vez. Cada vértice apilado se desapila exactamente una vez, y se apila una vez por arista más el inicio, así que hay E+1 apilados. Total: **O(V+E)** en tiempo.
- **Eventos:** hay 1 `start`, E `traverse`, E+1 `finish` y 1 `reverse`, es decir 2E+3 eventos, cada uno almacenado en O(1) (sin copiar la pila). Esto coincide con los 15 eventos del ejemplo con E = 6.
- **Memoria:** O(V+E) para listas de adyacencia, pila, salida y eventos.

El costo del renderizado de Manim es independiente y no entra en esta cota. Como cada arista debe visitarse al menos una vez para producir el ciclo, Ω(E) es una cota inferior: el algoritmo es asintóticamente óptimo.

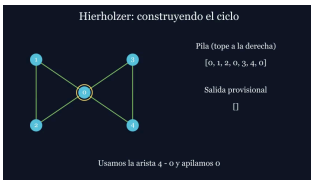
5. Evidencia



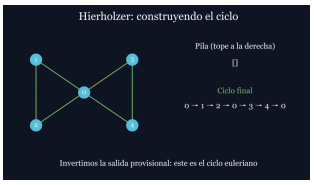
Grafo y grados (verificación)



Aristas consumidas; pila y salida



Retroceso: vértices pasan a la salida



Ciclo final

Capturas del video de avance. [captura del historial de commits].

6. Estado actual, conclusiones y trabajo restante

Conclusiones. La separación entre un motor en C++ que emite eventos y una escena de Manim que solo los reproduce garantiza que la animación refleje la ejecución real del algoritmo y permite cambiar de grafo sin tocar el código de animación. El uso explícito de la pila y del retorno de control hace visible la idea central de Hierholzer: los sub-ciclos pendientes se incorporan al retroceder, y la inversión final entrega el ciclo ordenado. Además, los tres casos de prueba validan tanto el caso exitoso como el caso de borde (sin ciclo).

Avance logrado: implementación completa del algoritmo con verificación de existencia, traza de eventos, escena animada de la ejecución sobre una instancia ilustrativa (42 s) y casos de prueba. **Pendiente para la entrega final (20 de noviembre):** escenas de motivación (Königsberg) e introducción conceptual, inclusión del caso de borde y del ejemplo con retroceso en el video, escena de complejidad, título y créditos con los integrantes, renderizado en alta resolución, y mantener commits graduales en el repositorio.

7. Referencias

- C. Hierholzer, “Ueber die Möglichkeit, einen Linienzug ohne Wiederholung und ohne Unterbrechung zu umfahren”, *Mathematische Annalen* 6, 30–32, 1873.
- L. Euler, “Solutio problematis ad geometriam situs pertinentis”, 1736 (puentes de Königsberg).
- Manim Community: https://docs.manim.community/en/stable/index.html · 3Blue1Brown: https://www.3blue1brown.com/ (referencia de estilo).
- Material del curso CS2023: diapositivas de teoría de grafos (UTEC, 2026-II).